Remy Bozung

Phone: (231) 335-1891 | Email: bozungremy@gmail.com | Linkedin: linkedin.com/in/remy-bozung/

Education

Computer Science Major, Philosophy and Math Minor Expected Graduation: Dec. 2027

Rhodes College | Memphis, TN GPA: 3.98/4.0

Relevant Coursework: Advanced Algorithms, Operating Systems, Dependent Types, Reinforcement Learning, Computer Networks, Parallel Systems

Technologies: C, C++, C#, Java, Python, CUDA, ARM Asm, Zephyr, Linux, Terraform (AWS), Git, CI/CD, I2C, SPI, UART

Work Experience

Lead Flight Software Researcher Feb. 2026 - Present

PEROVSAT | Memphis, TN

- Designed a modular, multithreaded flight software architecture in C++ using Zephyr, organizing application code into isolated routines that new students can develop independently
- Wrote device drivers and used compile-time dependency injection with Kconfig to select hardware, simulation, or mock implementations, enabling application development while drivers were incomplete
- Designed and built magnetic and power simulation subsystems
- Established a pull request process requiring documentation updates and unit tests before code is merged, preserving project knowledge and quality as students graduate
- Created a mission website to publish documentation and recruit student contributors

Software Development Intern -> ***Junior Software Developer*** May 2025 - Jan. 2026

eFulfillment Service | Traverse City, MI

- Designed and implemented our AI Sales Assistant, which integrates with CRM software to provide real-time searches, analysis, and graphing of sales data
- Built an AWS Prometheus Module, which scans ECS instances for metrics
- Built an AWS Synthetic Canary Terraform module to streamline continuous testing of AWS deployments
- Built and deployed Github Workflow to auto-generate documentation and diagrams of AWS infrastructure

Projects

Gate-Lang Apr. 2026

Utilized: C++, ProtoBuf, Compiler Design, PEG Parsing, CMake

Overview: A component-based Hardware Description Language. Parses and compiles to a ProtoBuf object, then used in other applications like the simulator or grapher.

Pure Wordle Jan. 2026

Utilized: C++, OpenMP, AVX2, GoogleTest, Slurm, Perf Profiling, CMake

Overview: An optimized, parallelized dynamic-programming Wordle solver. Uses sharded hashmaps to minimize lock contention, and lookup tables with a custom std::bitset implementation to enable vectorized operations.